Writing skills *for Claude Code* agents.

Guideline, features and templates

A practical guideline for writing, configuring and using agents and skills

Canache Gabriel
QA Engineer
Centric Romania

Version 1.0 · October 2026

centric

Contents

1. Introduction **4**

1.1 Agent and skill 4

2. Folder structure **5**

3. Skill features **6**

F1 Description contract 6
F2 Role in the flow 6
F3 Context block 7
F4 Sources of truth 7
F5 References table 7
F6 Output templates 8
F7 Scripts and helpers 8
F8 Numbered workflow 8
F9 Verification 9
F10 Bounded retries 9
F11 Final report 9
F12 Human approval gate 10
F13 Idempotent write-back 10
F14 Guardrails 10
F15 Lessons 11
F16 Worked example 11

4. Templates **12**

4.1 SKILL.md 12
4.2 Agent file 13
4.3 Reference file 13
4.4 Output file with an approval gate 13

5. Agent configuration and access **14**

5.1 Where agents are defined 14
5.2 Configuration fields 14
5.3 How access works 15
5.4 Giving an agent access to an MCP server 15
5.5 Giving access to files and folders 16
5.6 Giving access to commands 17

5.7 Permission modes 17
5.8 Where settings live 17
5.9 Example: a locked-down reporting agent 18

6. Using agents in Claude Code **19**

6.1 Adding an MCP server 19
6.2 Calling an agent 20

7. Checklist **21**

1. Introduction

How to write an agent and a skill, how to configure them, and how to use them in Claude Code.

This guideline describes a practical way to build AI agents in Claude Code. It covers three things: how an agent and its skill are written, how they are configured and given access to tools, and how they are used day to day.

The recommendations come from a working multi-agent solution that is used in real projects. The patterns that proved useful there were generalised into features and templates, so they can be reused for any kind of agent.

1.1 Agent and skill

	Agent	Skill
File	`.claude/agents/<name>.md`	`.claude/skills/<name>/SKILL.md`
Defines	**What it may use:** tools, model, permission mode	**How it works:** inputs, workflow, rules
Runs	In its own context, returns a report	Inside the session or agent that loads it
Called with	`@agent-<name> <task>`	`/<name>` or picked from its description

Pair every agent with a skill of the same name: the agent restricts the tools, the skill explains the work.

2. Folder structure

Where agents, skills and their supporting files live in a project.

Recommended layout

```
<project>/
├── .claude/
│   ├── agents/                       # one file per agent: tools, model, skill
│   │   └── <agent-name>.md
│   ├── skills/                       # one folder per skill
│   │   └── <skill-name>/
│   │       ├── SKILL.md              # entry point
│   │       ├── references/           # loaded only when a step needs them
│   │       │   ├── <topic>.md
│   │       │   └── <output>-template.md
│   │       └── scripts/              # deterministic checks
│   │           └── check-<thing>.mjs
│   ├── settings.json                 # team permissions (committed)
│   └── settings.local.json           # personal permissions (git-ignored)
├── .mcp.json                         # MCP servers shared by the team
├── CLAUDE.md                         # project context, loaded automatically
├── lessons/                          # one lessons file per skill
│   └── <skill-name>-lessons.md
├── tools/                            # helpers for sensitive actions
│   └── <helper>.mjs
└── .secrets/                         # local credentials (git-ignored)
```

Item	Role
`.claude/agents/`	The **capabilities** of each agent. Short files, frontmatter only.
`.claude/skills/`	The **procedures**. `SKILL.md` plus `references/` and `scripts/`.
`.claude/settings.json`	Permission rules for files, commands and MCP tools (chapter 5).
`.mcp.json`	The external tools the agents can call: tracker, database, API, browser.
`CLAUDE.md`	Project context that every session and agent receives automatically.
`lessons/`	What each agent learned from human corrections. Read at the start of every run.
`tools/`	Helpers that do sensitive work (for example signing in) so the model never sees secrets.
`.secrets/`	Local credentials, never committed and never readable by the agents.

Start Claude Code from the project folder

Agents, skills, settings and `.mcp.json` are discovered from the folder Claude Code is started in. If an agent does not appear when you type `@`, check where you started `claude`.

3. Skill features

Sixteen building blocks of a good skill. Pick the ones your skill needs and copy the template.

Group	Features
Identity and context	F1 Description contract · F2 Role in the flow · F3 Context block · F4 Sources of truth
Knowledge loading	F5 References table · F6 Output templates · F7 Scripts and helpers
Procedure	F8 Numbered workflow · F9 Verification · F10 Bounded retries · F11 Final report
Collaboration	F12 Human approval gate · F13 Idempotent write-back
Safety and learning	F14 Guardrails · F15 Lessons · F16 Worked example

Identity and context

F1 Description contract

The frontmatter `description` is one dense sentence that says what the skill reads, what it produces, the rule that matters most and where the result can be found.

Why: Claude chooses skills and agents by this text, and it is the first thing a reviewer reads.

Template

```
---
name: <skill-name>
description: <Reads what> from <source>, <produces what> in <location>,
  <the key rule>, and reports the result in <task / file>.
---
```

F2 Role in the flow

The first paragraph places the skill in the flow: which step comes before, which comes after, and what it deliberately does not do.

Why: Prevents overlap between agents and stops a skill from doing another skill's job.

Template

```
# <Skill Title>

The <Skill Title> is the step after `<previous-skill>`. It <main job>.
It does not <what belongs to another skill>: that is `<other-skill>`'s job.
```

F3 Context block

A few lines (or a small table) with the system the agent works on, the tools it talks to and what is already loaded in context.

Why: The agent starts with the facts it needs and does not waste tokens re-reading files.

Template

```
**System:** <app name> - API <url>, UI <url>, database <name>.
**Tracker:** <organisation> / <project>.
`CLAUDE.md` is already in your context - do not Read it again.
Large files: grep the `## ` heading, then Read with offset / limit.
```

F4 Sources of truth

List the sources the skill may use and which one wins when they disagree. For testing or review skills, forbid looking at the implementation.

Why: Keeps results unbiased: the expected behaviour comes from the requirements, not from how the code was written.

Template

```
Expected results come only from: <requirements>, <API contract>, <project docs>.
Never open source code, tests, pull requests, commits or git history.
When the requirements and a document disagree, the requirements win;
list the difference as a gap. Anything only code could answer -> Open Questions.
```

Knowledge loading

F5 References table

Details live in `references/`. A table in `SKILL.md` says which file to load at which step.

Why: Keeps `SKILL.md` short (about 120 to 200 lines) and loads knowledge only when it is needed.

Template

```
## References (load only when the step needs it)

| File                                  | When                                 |
|---------------------------------------|--------------------------------------|
| [references/<topic>.md](...)          | Step 1 - how to read the input       |
| [references/<template>.md](...)       | Step 4 - output template             |
| [../<other-skill>/references/<x>.md]  | Step 2 - shared with <other-skill> |
```

F6 Output templates

Every file or item the skill produces has a template in `references/`, with fixed headings and field names.

Why: Outputs look the same on every run, so humans can review them quickly and the next agent can parse them.

Template

```
# references/<output>-template.md

## Title
<Where>: <what happens>             (max 80 characters)

## Steps
1. <action>
2. <action>

## Expected / Actual
- Expected: <result> (source: <requirement>)
- Actual:   <result>

## Evidence
<screenshot path, response excerpt, log line>
```

F7 Scripts and helpers

Checks that must be exact go into `scripts/`. Sensitive actions such as signing in go into a helper in `tools/` that prints only a short result.

Why: A script gives the same answer every time; a helper keeps passwords and tokens out of the model's context.

Template

```
// scripts/check-<thing>.mjs
// Checks that <every entry in X still exists in Y>.
// Usage: node .claude/skills/<skill>/scripts/check-<thing>.mjs
// Exit 0 = in sync, 1 = stale entries (listed), 2 = could not read inputs.

// tools/<helper>.mjs
// Reads credentials from .secrets/<file>.env and signs in.
// Prints a one-line result - never the password or the session token.
```

Procedure

F8 Numbered workflow

The procedure is a numbered list starting at 0. Each step does one thing, names the reference it needs and states when to stop.

Why: The agent always knows where it is, and a human can follow or resume the run.

Template

```
## Workflow
0. **Lessons.** Read the lessons file and apply it.
1. **Collect.** <read inputs>. Nothing found -> say so, stop.
2. **Understand.** <read sources> (references/<x>.md).
3. **Do.** <main action>.
4. **Verify.** <prove the result> (see F9).
5. **Report** to the human (see F11).
```

F9 Verification

A step that proves the result before anything is reported: the output was re-read against the rules, the work really happened, nothing was left behind.

Why: “Done” means verified, not just executed.

Template

```
## Verify
- Re-read every output once against the Guardrails before sending it.
- The result really happened: <e.g. tests found = expected count, none skipped,
  the assertion fails when the expected value is changed>.
- Nothing is left behind: <e.g. rows carrying the run tag = 0>.
- A result counts only when <precondition, e.g. the app was healthy>.
```

F10 Bounded retries

A fixed number of attempts to fix a failure, with a clear rule on what may and may not change.

Why: Stops endless loops and stops the agent from “fixing” a problem by hiding it.

Template

```
## When something fails
- At most 3 fix attempts per failure; each attempt is followed by a re-run.
- A fix may change only HOW the step runs (e.g. a locator, a wait, the setup),
  never WHAT is checked (expected value, assertion strength).
- If the failure shows a real defect: stop fixing, leave it failing, report it
  with the evidence.
- After the third attempt: leave it as it is and move on.
```

F11 Final report

The last step tells the human, in a fixed shape, what was done, what was skipped and what needs their action.

Why: The human can decide the next step in seconds.

Template

```
## Report (to the human)
| Item | Result | Created id | Notes |
|------|--------|------------|-------|
Then list:
- already done (skipped, with id)
- skipped (with reason) and invalid (with what to fix)
- blocked items and the human action each one needs
```

Collaboration

F12 Human approval gate

Outputs that leave the local workspace wait for a human decision written in the file. The next agent acts only on approved items.

Why: A human stays in control of everything that reaches an external system.

Template

```
## Reviewer Decision
Status: PENDING            <!-- human writes APPROVED or REJECTED -->
Reviewer note:

# In the next skill's Guardrails:
- Act only on items whose Status is APPROVED.
- Never act on PENDING, REJECTED or an unclear status; never change a decision.
```

F13 Idempotent write-back

Before creating anything external, check for a duplicate. After creating it, write its id back into the source file.

Why: Running the same command twice never creates the same item twice.

Template

```
1. Skip items that already carry `<id field>:` in the file.
2. Duplicate check: search the external system for `<unique tag or text>`.
   Found -> do not create; write its id back; report "already done".
3. Create the item.
4. Write back: `<id field>: <new id>` and `<date field>: <date>` in the file.
   If create succeeded but a follow-up step failed, still write back and report it.
```

Safety and learning

F14 Guardrails

A closing list of rules that override everything else in the skill, including lessons.

Why: The few things that must never happen are stated once, clearly, in one place.

Template

```
## Guardrails
- **One write target:** the only thing you may create or change is <...>.
  Everything else is read-only.
- **Credentials:** never read <secrets folder> or <.env>, never print a password
  or token.
- **Data:** synthetic test data only (e.g. `@example.test`); never touch
  records you did not create.
- **Transcribe, do not invent:** every fact comes from the source; gaps are
  listed, never guessed.
- **Hands off:** never edit <application folders>, never reset the environment,
  never commit or push.
```

F15 Lessons

Each skill reads its lessons file at step 0 and appends to it only when a human corrects it.

Why: The agent improves from feedback without the skill itself being rewritten after every run.

Template

```
# lessons/<skill>-lessons.md
Rules learned from user corrections. Read before every run. Newest last.

## YYYY-MM-DD - <short title>
- Lesson: <the rule>
- Why: <what went wrong or what the user said>
- How to apply: <when and how it changes the work>

# In SKILL.md:
A lesson overrides the general guidance, never the Guardrails.
Do not add lessons you inferred yourself.
```

F16 Worked example

The skill ends with one realistic run: the command, what the agent finds, what it writes and what it reports.

Why: One concrete example explains the skill faster than any rule.

Template

```
## Example
User: "<Command> 123."

The agent finds <n> items: <a> are APPROVED, <b> REJECTED. It finds no
duplicates, creates <a> items, writes their ids back into the files,
and reports: <a> created, <b> skipped.
```

4. Templates

The features combined into ready-to-copy files.

4.1 SKILL.md

.claude/skills/<skill-name>/SKILL.md

```
---
name: <skill-name>
description: <Reads what>, <produces what>, <key rule>, and reports the result
  in <task / file>.
---

# <Skill Title>

The <Skill Title> is the step after `<previous>`. It <main job>. It does not <...>.

**System:** <urls>. **External tools:** <tracker, database, ...>.
CLAUDE.md is already in your context - do not Read it again.

## References (load only when the step needs it)
| File                    | When        |
|-------------------------|-------------|
| references/<a>.md       | Step 1      |
| references/<b>.md       | Step 3      |

## Inputs
- `<Command> <ID>` - <default scope>
- `<Command> <ID> dry run` - read and plan only

## Tools
- `<server>` - read: <...>. Write ONLY: <...>. Nothing else.
- `Write` / `Edit` - only <folder> and the lessons file.

## Workflow
0. Read the lessons file.
1. Collect <inputs>. None -> say so, stop.
2. <step> (references/<b>.md)
3. Verify: re-read against the Guardrails; <proof>.
4. Report to the human.

## Guardrails
- The only thing you may create or change is <...>.
- No credentials; synthetic data only; never edit the application; never commit.

## Lessons
Read `lessons/<skill>-lessons.md` first. Append only when a human corrects you.

## Example
User: "<Command> 123." -> <what it finds, writes and reports>
```

4.2 Agent file

.claude/agents/<skill-name>.md

```
---
name: <skill-name>
description: <same contract as the skill description>
tools:
  - Read
  - Glob
  - Grep
  - Edit
  - Skill
  - mcp__<server>                # whole server, or mcp__<server>__<tool>
model: opus
effort: high
permissionMode: acceptEdits
skills:
  - <skill-name>
---
```

4.3 Reference file

.claude/skills/<skill-name>/references/<topic>.md

```
# <Topic> in <tool>

<One paragraph: what this file covers and the only thing the skill may change.>

Tools (`<server>` MCP):
- read:  `<tool>` (`get`, `list`)
- write: `<tool>` (`create`, `update`)

## 1. Find the input
## 2. Find or create <the item>
## 3. Write <the result>
## 4. Confirm and report
```

4.4 Output file with an approval gate

<output-folder>/<ITEM-01>-<slug>.md

```
---
source: <ID>
id: <ITEM-01>
---
# <ITEM-01> - <Where>: <what happens>

## Steps
1. <action>

## Expected
<result> (source: <requirement>)

## Actual
<result>

## Evidence
<screenshot / response excerpt / log line>

## Reviewer Decision
Status: PENDING        <!-- APPROVED or REJECTED -->
```

5. Agent configuration and access

How an agent is set up, and how to give it access to MCP servers, files and commands.

5.1 Where agents are defined

An agent is a Markdown file with a YAML block at the top. Claude Code loads agents from two places:

Location	Available in	Use it for
`.claude/agents/<name>.md`	This project, for everyone who clones it	Team agents
`~/.claude/agents/<name>.md`	All your projects, only for you	Personal helpers

You can write the file by hand or run `/agents`, which walks you through name, description, tools and model, and lets you edit an existing agent later. When a project agent and a personal agent have the same name, the project one is used.

5.2 Configuration fields

.claude/agents/<agent-name>.md

```
---
name: <agent-name>                    # id used by @agent-<agent-name>
description: <Reads what>, <produces what>, <key rule>, and reports
  the result in <task / file>.
tools:                                # allowlist; omit it to inherit every tool
  - Read
  - Glob
  - Grep
  - Edit
  - Skill
  - mcp__<server>                     # all tools of one MCP server
model: opus                           # opus | sonnet | haiku | inherit
effort: high                          # reasoning effort
permissionMode: acceptEdits           # how permission prompts behave
skills:                               # skills preloaded into the agent
  - <skill-name>
---
<optional system prompt - can stay empty when the skill holds the procedure>
```

Field	What it controls	Tip
`name`	The agent's id; `@agent-<name>` uses it	Same name as the skill
`description`	When Claude delegates to the agent	Say what it reads, writes and reports
`tools`	Which tools the agent may use at all	Give only what the skill needs
`disallowedTools`	Tools removed from an otherwise broad list	Remove risky tools of a server you grant whole
`model`	Model alias, or `inherit` from the session	Use a stronger model for judgement-heavy agents
`effort`	Reasoning effort	`high` for planning and analysis
`permissionMode`	Whether edits / commands ask for approval	See 5.7
`skills`	Skills loaded into the agent's context	One skill per agent keeps it focused
Body	The agent's system prompt	Keep it short or empty when a skill exists

Least privilege

Give each agent only the tools its job needs. An agent that only reads files and writes to one external system does not need `Write`, `Bash`, a browser or a database.

5.3 How access works

A tool call goes through several layers. It succeeds only if every layer lets it through, so you can grant access broadly in one layer and narrow it in another.

#	Layer	Where	Question it answers	Enforced
1	Server definition	`.mcp.json`	Does the MCP server exist?	Yes
2	Server enabled	Approval prompt / `enabledMcpjsonServers`	Is the server switched on in this project?	Yes
3	Agent tools	`tools:` / `disallowedTools:` in the agent file	May this agent use the tool?	Yes
4	Permission rules	`allow` / `ask` / `deny` in `settings.json`	Is this specific call allowed, asked or blocked?	Yes
5	Skill text	Tools and Guardrails in `SKILL.md`	How and when should the agent use it?	No (instruction)

Rule of thumb

Explain a rule in the skill (layer 5) so the agent understands it, and enforce the same rule in layer 3 or 4 so it holds even when the agent gets it wrong.

5.4 Giving an agent access to an MCP server

Step 1. Define the server

Add it to `.mcp.json` (or with `claude mcp add`, see chapter 6). Its tools are then named `mcp__<server>__<tool>`; `/mcp` lists the exact names.

Step 2. Enable it for the project

.claude/settings.json (or approve the servers when Claude Code asks)

```
{
  "enabledMcpjsonServers": ["<server-a>", "<server-b>"],
  "disabledMcpjsonServers": ["<server-c>"]
}
```

Step 3. Grant it in the agent file

.claude/agents/<name>.md

```
tools:
  - mcp__<server-a>                      # every tool of server-a
  - mcp__<server-b>__<read-tool>         # only these two tools
  - mcp__<server-b>__<write-tool>        #   of server-b
disallowedTools:
  - mcp__<server-a>__<risky-tool>        # or: grant whole, remove a few
```

Step 4. Allow, ask or block individual tools

.claude/settings.json

```
{
  "permissions": {
    "allow": ["mcp__<server-a>", "mcp__<server-b>__<read-tool>"],
    "ask":   ["mcp__<server-b>__<write-tool>"],
    "deny":  ["mcp__<server-a>__<risky-tool>"]
  }
}
```

Step 5. Describe the allowed use in the skill

SKILL.md, Tools section

```
- `<server-b>` - read: `<read-tool>`. Write ONLY: `<write-tool>` (create)
  for <item type>. Nothing else. Auth error -> ask the human to run /mcp.
```

5.5 Giving access to files and folders

File access is controlled by which file tools the agent has, by `Read(...)` / `Edit(...)` rules, and by which folders Claude Code can see.

Need	Agent tools	Rule in settings.json
Read and search files	`Read`, `Glob`, `Grep`	`Read(<glob>)`
Change existing files	`Edit`	`Edit(<glob>)`
Create new files	`Write`	`Edit(<glob>)` (covers writes too)
Never touch a path	(none)	`deny`: `Read(<glob>)` and `Edit(<glob>)`

By default Claude Code only works inside the folder it was started from. If an agent needs a folder outside it, for example documentation one level up, add that folder:

.claude/settings.json

```
{
  "permissions": {
    "additionalDirectories": ["../docs/"],
    "allow": [
      "Read(../docs/**)",
      "Edit(./<output-folder>/**)",
      "Edit(./lessons/**)"
    ],
    "deny": [
      "Read(./.secrets/**)",
      "Read(./**/.env)",
      "Edit(../<application-folder>/**)",
      "Edit(../docs/**)"
    ]
  }
}
```

For a one-off session you can also run `claude --add-dir ../docs` or type `/add-dir ../docs` inside the session. Paths starting with `./` are relative to the folder you started in, `~/` to your home folder, and `//` marks an absolute path.

5.6 Giving access to commands

An agent can run shell commands only if `Bash` is in its tools. Then limit which commands with `Bash(...)` rules, matching the start of the command:

.claude/settings.json

```
{
  "permissions": {
    "allow": [
      "Bash(node tools/<helper>.mjs:*)",
      "Bash(npm test:*)"
    ],
    "deny": [
      "Bash(git push:*)",
      "Bash(rm -rf:*)"
    ]
  }
}
```

5.7 Permission modes

Mode	Behaviour	When to use it
`default`	Asks the first time each tool is used	New or untrusted agents
`acceptEdits`	File edits run without asking; other tools follow the rules	Agents that write files in a known folder
`plan`	Read-only: it can explore and plan, not change anything	Reviews and analysis
`bypassPermissions`	Skips all prompts	Avoid, especially with access to external systems

A `deny` rule always wins, whatever the mode. That is why secrets and application folders belong in `deny`, not only in the skill.

5.8 Where settings live

File	Scope	In git?
`.claude/settings.json`	Team rules for this project	Yes
`.claude/settings.local.json`	Your personal rules for this project	No
`~/.claude/settings.json`	Your rules for every project	No

- **Precedence between rules:** `deny` > `ask` > `allow`.
- `/permissions` shows the rules in effect and lets you add or remove them.
- `/mcp` shows which servers are connected and their tool names.

5.9 Example: a locked-down reporting agent

A typical case: an agent that reads approved items from local files, writes one line back into each file and creates the items in an external tracker. This is a complete set-up for that job.

```
# .claude/agents/<reporter>.md
tools:
  - Read
  - Glob
  - Grep
  - Edit
  - Skill
  - mcp__<tracker>__<get-item>
  - mcp__<tracker>__<search-items>
  - mcp__<tracker>__<create-item>
permissionMode: acceptEdits

// .claude/settings.json
{
  "permissions": {
    "allow": ["Edit(./<output-folder>/**)", "Edit(./lessons/**)"],
    "ask":   ["mcp__<tracker>__<create-item>"],
    "deny":  ["Read(./.secrets/**)", "Read(./**/.env)"]
  }
}
```

With this, the agent cannot create files, run commands, open a browser or query a database, and Claude Code asks before every item it creates. The skill still explains the rest: which items count as approved, and that it may only write back the new id.

6. Using agents in Claude Code

Adding MCP servers and calling an agent.

6.1 Adding an MCP server

Option A: edit `.mcp.json` in the folder Claude Code starts from. It is shared with the team through git.

.mcp.json

```
{
  "mcpServers": {
    "<server-name>": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "<package>@latest", "<arg>"],
      "env": { "<TOKEN_NAME>": "${<ENV_VARIABLE>}" }
    },
    "<remote-server>": {
      "type": "http",
      "url": "https://<host>/mcp"
    }
  }
}
```

- Secrets come from environment variables through `${VAR}`; never write them in the file.
- On Windows, wrap npx: `"command": "cmd"`, `"args": ["/c", "npx", …]`.
- Tool names become `mcp__<server>__<tool>`; these names are used in agent `tools:` lists and permission rules.

Option B: use the CLI.

```
# shared with the team (writes .mcp.json)
claude mcp add --scope project <server-name> -- npx -y <package>@latest

# with an environment variable
claude mcp add --scope project <server-name> -e <TOKEN_NAME>=<value> -- <command> <args>

# remote HTTP server
claude mcp add --transport http <server-name> <https-url>

claude mcp list                 # all servers and status
claude mcp get <server-name>    # details
claude mcp remove <server-name>
/mcp                            # inside a session: status and re-authentication
```

Scope	Stored in	Visible to
`local` (default)	`~/.claude.json`	Only you, this project
`project`	`.mcp.json`	Everyone who clones the repository
`user`	`~/.claude.json`	Only you, all projects

6.2 Calling an agent

```
@agent-<agent-name> <command> <arguments>

# examples
@agent-<planner> Plan Story 123
@agent-<reporter> Report items 123 dry run
```

- Type `@`, pick the agent (shown as `agent-<name>`), then write the task in the format the skill's Inputs section defines.
- Keep commands short and consistent: a verb, an id, and optional variants such as a subset of ids or `dry run`.
- The agent runs in its own context and returns a report to your session.
- Plain language also works (“use the planner agent to plan story 123”), but the `@` mention leaves no doubt which agent runs.

Command	What it does
`@agent-<name> <task>`	Runs that agent with the task
`/agents`	Lists, creates and edits agents
`/<skill-name> <args>`	Runs a skill directly in the current session
`/mcp`	Checks the MCP servers and their tools
`/permissions`	Shows and edits the permission rules

7. Checklist

Review every new agent and skill against this list before running it.

1. The description says what the skill reads, produces and where the result goes (F1), and matches the agent's description.
2. The first paragraph places the skill in the flow and says what it does not do (F2).
3. The sources of truth are listed, with which one wins on conflict (F4).
4. `SKILL.md` stays under about 200 lines; details are in `references/`, each tied to a step (F5).
5. Every output has a template (F6); exact checks are scripts (F7).
6. The workflow is numbered from 0, with stop conditions (F8) and a verification step (F9).
7. Retries are bounded (F10) and the run ends with a report (F11).
8. Anything that leaves the workspace passes a human gate (F12) and is written back (F13).
9. Guardrails (F14), a lessons file (F15) and a worked example (F16) are present.
10. The agent's `tools:` list contains only what the skill needs (5.2).
11. What the skill promises is enforced in `settings.json` (5.3 to 5.6).
12. The first run is a dry run.

Canache Gabriel · QA Engineer · Centric Romania